

Governed Autonomy as a Manufactured Artifact

The Deletion of Implementation as a Phase Transition
in the Engineering of Autonomous Software Agents

Sean Chatman
AutoFDE Laboratory
`sean@chatmangpt.com`

September 2026

Abstract

Autonomous software agents must allocate finite exploration resources under uncertainty, and every existing family of allocation policies—greedy selection, ε -greedy exploration, confidence bounds (UCB1), and predictor–confidence upper-confidence trees (PUCT)—optimizes expected performance while remaining structurally incapable of governance: their outputs are non-deterministic across platforms, non-conserving under discretization, silently degrading under failure, and invisible to after-the-fact audit. This thesis argues and demonstrates that the binding constraint on trustworthy agent autonomy is not exploration performance but the *auditability of consequential decisions*, and that removing this constraint *structurally*—rather than mitigating it—constitutes a technological phase change in the sense established by the steam engine and the transistor: each removed an entire failure class of its predecessor, and each thereby unlocked composition (the factory system; integrated circuits) that the old regime could not reach at any price.

The demonstrated artifact is a single certified allocation kernel (`bcinr-cmca`, branch-less, cyclomatic complexity $CC = 1$, Q16.16 fixed-point) that serves every host in a polyglot ecosystem bit-identically: in-process through WebAssembly on both CPython (via `wasmtime`) and the BEAM virtual machine (via `wasmex`), with agreement pinned by golden fixtures whose values are exact binary fractions. The work’s methodological core is a *falsification ledger*: the thesis deliberately reports and cites the deletion of its own weaker claims—a circular comparative benchmark, a set of “theorems” that held for any softmax, a notational equivalence proof, and ultimately the entire local reference implementation of the allocation measure (removed in commit `c1d04ffc`). What survives falsification is the governance envelope: deterministic replay, conservation by construction, typed refusal, and content-bound receipts streamed into Object-Centric Event Log (OCEL 2.0) models where they are conformance-checked and counterfactually mutated.

The empirical record includes an honest negative finding: the certified kernel’s compiled lens policy is substantially *flatter* than the deleted tunable softmax (a dead-end candidate receives a measured 16.9% share where the softmax at high inverse temperature gave it none), a trade-off we document rather than tune away. The phase-change claim is argued structurally—failure-class removal, composition unlock, and a compounding standard interface—and scoped honestly: this is a laboratory demonstration of structure, not a demonstrated historical shift. Falsifiers for the analogy are stated explicitly.

Contents

1	Introduction	3
1.1	The problem that would not go away	3
1.2	The thesis statement	3
1.3	Contributions	4
1.4	Why this needed a laboratory, not just a paper	4
2	Background and Related Work	5
2.1	Allocation under uncertainty	5
2.2	Deterministic arithmetic and certified kernels	5
2.3	Object-centric process mining	5
2.4	Hierarchical planning and agent execution	6
2.5	The industrial analogues	6
3	The Governance Envelope	7
3.1	What CMCA is, honestly	7
3.2	The four laws	7
3.3	What the envelope buys, and what it costs	8
4	The Single-Kernel Law	9
4.1	From one engine per host to one kernel for all hosts	9
4.2	The WebAssembly deployment	9
4.3	Bit-identity across hosts	10
4.4	The honest finding: flatness	10
5	Process Law and Counterfactual Conformance	12
5.1	Allocations as events	12
5.2	Conformance and counterfactual mutation	12
5.3	The limitation we declare	12
6	The Falsification Ledger	13
6.1	Deletion as a scholarly act	13
6.2	The autopsies	13
6.2.1	The circular benchmark	13
6.2.2	The theorems that held for every softmax	13

6.2.3	The notational equivalence	14
6.2.4	The reference implementation itself	14
6.3	What the ledger buys	14
7	The Phase-Change Analysis	15
7.1	A model of phase change	15
7.1.1	Steam	15
7.1.2	The transistor	15
7.2	The mapping	16
7.3	Disanalogies, stated	16
7.4	Falsifiers	17
8	Conclusion	18
A	Artifact and Verification Map	19
B	The Deleted Claims, For the Record	20

Chapter 1

Introduction

1.1 The problem that would not go away

Software agents that act on the world must decide, repeatedly and under uncertainty, how to divide finite resources—compute ticks, memory, verification depth, consequence budget—across candidate courses of action. The classical answers are well understood and theoretically characterized: greedy exploitation, ε -greedy exploration, UCB1’s optimism in the face of uncertainty [1], and PUCT-style search [12]. Each is a performance instrument. None is a governance instrument. Their outputs cannot in general be replayed bit-identically across platforms; their budgets leak under discretization; their failures degrade silently through fallbacks; and their decisions leave no durable, checkable trace.

For toy systems this does not matter. For *consequential* autonomy—agents whose actions carry authority, cost, or risk—it is the binding constraint. An unauditable allocation is an unaccountable one, and an unaccountable allocation cannot be delegated, chained, or composed across trust boundaries. The hypothesis of this thesis is that this constraint, not exploration suboptimality, is what currently prevents the composition of autonomous agents into large, trustworthy systems.

1.2 The thesis statement

Principle 1 (Thesis statement). *When the allocator of an autonomous agent’s consequential resources is manufactured as a single certified artifact—deterministic across hosts, conserving by construction, refusing by type rather than degrading silently, and emitting content-bound receipts into a process-mining event log—the failure class of divergent, silent, unverifiable implementation is removed structurally. This removal unlocks composition (polynomial numbers of hosts, long chains of receipted delegation, conformance-checked lifecycles) that per-implementation governance cannot reach at any tuning budget, in the same structural sense in which the steam engine removed the biological-power failure class and the transistor removed the filament failure class.*

Three clarifications keep this claim honest. First, the analogy is *structural*, not historical: we demonstrate a mechanism, not a societal shift. Second, at the moment of transition the

new regime may be *worse* on legacy metrics—Newcomen’s engine was thermally inefficient, the first transistors were not faster than tubes, and our certified kernel is flatter than a tuned softmax; Chapter 7 argues that this is characteristic, not incidental. Third, the claim is falsifiable, and Chapter 7 states what would falsify it.

1.3 Contributions

1. **The governance envelope, formalized** (Chapter 3): a precise statement of the four laws a consequential allocator must satisfy—deterministic replay, conservation, typed refusal, receipt—with an explicit demotion of the allocation *mathematics* to standard constructions (escort measures over multifractal cascades) and an account of why the envelope, not the measure, carries the contribution.
2. **The single-kernel law** (Chapter 4): an architecture and a verified deployment in which one certified Rust kernel is the only implementation of the allocation measure in a polyglot ecosystem, served in-process via WebAssembly to CPython and the BEAM, with bit-identical outputs pinned by cross-host golden fixtures. The local reference implementation was deleted outright, and its deletion is pinned by a test that makes reintroduction a structural type error.
3. **Process-law conformance for allocations** (Chapter 5): allocation plans as first-class OCEL 2.0 events, checked by alignments against discovered process models and adversarially mutated by a counterfactual battery.
4. **The falsification ledger as method** (Chapter 6): a documented, commit-cited record of claims made, claims falsified, and claims survived—including the autopsy of our own circular benchmark. We propose deletion-with-receipt as a first-class scholarly act.
5. **A phase-change analysis** (Chapter 7): the constraint-removal model of technological phase transitions, applied with explicit disanalogies and falsifiers.

1.4 Why this needed a laboratory, not just a paper

Every claim above lives or dies on artifacts: a kernel that exists, hosts that agree bit-for-bit, tests that fail when the law is broken, and a commit history in which the negative results are visible. The thesis is therefore written *from* the artifact chain summarized in Appendix A; where a number appears in the text, it was measured, and the measurement is reproducible from the cited commit.

Chapter 2

Background and Related Work

2.1 Allocation under uncertainty

The multi-armed bandit literature establishes the performance frontier for sequential allocation: UCB1’s logarithmic regret [1], PUCT’s confidence-driven search [12], and the general treatment in Sutton and Barto [13]. This literature optimizes reward under a trust model in which the *implementor* of the policy is trusted absolutely. The present work assumes the opposite trust model—the policy’s decisions must be verifiable by parties who did not implement it—and therefore asks a question the bandit literature does not pose: what is the *decision artifact* that a third party can replay, conserve, and audit?

2.2 Deterministic arithmetic and certified kernels

Floating-point divergence across platforms is classical, well-documented [3], and fatal to bit-identical replay. The standard remedy is fixed-point arithmetic; the kernel at the center of this thesis uses Q16.16 (sixteen integer bits, sixteen fractional bits, resolution 2^{-16}) with saturating operations whose truncation is one-sided. The deeper tradition is certified code: kernels constrained to cyclomatic complexity $CC = 1$ — straight-line, branchless, allocation-free—inherit constant-time execution and side-channel resilience as theorems of their shape rather than properties of their testing. MacKenzie’s study of mechanized proof [5] frames the sociological point this thesis operationalizes: trust in computation migrates from persons to artifacts only when the artifacts carry their own evidence.

2.3 Object-centric process mining

The Object-Centric Event Log standard (OCEL 2.0) [8, 10] records events tied to multiple objects with qualifiers, enabling process conformance over entities rather than flattened traces. Object-centric process querying extends this to declarative structural laws over event logs [9]. The present work uses OCEL 2.0 as the *durably checkable substrate* for governance: an allocation that becomes an event with a content-bound hash can be

conformance-checked, counterfactually mutated, and replayed long after the process that produced it is gone.

2.4 Hierarchical planning and agent execution

FOND planning and HDDL hierarchical task networks provide the exploration structures over which allocation happens; GymAct-style governed simulation provides execution. These are the consumers of the allocator and are treated as given, cited in Appendix A as artifact rather than surveyed here as literature.

2.5 The industrial analogues

Landes’ account of the industrial revolution [4], Rosenberg’s studies of technological change [11], and Mokyr’s economic history of invention [6] establish the pattern this thesis borrows: general-purpose technologies transform when a *constraint class* (fuel availability at the minehead, component reliability) is removed and improvement effort compounds onto the new regime. The transistor’s own record—Bardeen and Brattain’s 1948 demonstration [2] and Moore’s 1965 composition argument [7]—supplies the cleanest instance: the initial win was reliability (no filament), and the compounding came from composition (integration). Chapter 7 develops this.

Chapter 3

The Governance Envelope

3.1 What CMCA is, honestly

Chatman Multifractal Cascade Allocation (CMCA) names the following construction. Given candidate branches $c_1, \dots, c_N$ with option entropy $\Omega(c_i)$, historical yield $Y(c_i)$, and cost $C(c_i)$, define the salience density

$$S(c_i) = \frac{\Omega(c_i) Y(c_i)}{\max(C(c_i), \varepsilon)}, \quad (3.1)$$

and allocate mass by a multifractal escort cascade: at each parent with children, the local escort distribution at lens q is

$$L_q(c_i | p) = \frac{m(c_i)^q}{\sum_j m(c_j)^q}, \quad m(c) = e^{S(c)}, \quad (3.2)$$

and a node’s consequence mass is the product of local escorts down its root path. Setting $m = e^S$ makes (3.2) a Gibbs–Boltzmann softmax at inverse temperature $\tau = q$; the “multifractal” content is the per-level lens vector $(q_0, q_1, \dots)$, which the certified kernel compiles from an ontology-declared policy.

We state plainly, as a matter of scholarly hygiene and as the first entry of the falsification ledger (Chapter 6): *the mathematics is standard*. Equation (3.1) is a heuristic payoff density; equation (3.2) is the textbook escort distribution of multifractal analysis; the one-line identity $dH/d\tau = -\tau \text{Var}(S) \leq 0$ for the softmax’s entropy is an exponential-family fact. An earlier iteration of this work presented these as “theorems” of CMCA; audit showed they hold for *any* softmax allocator, and the corresponding test file was deleted (commit-recorded in Appendix A). What survives audit, and what this chapter exists to formalize, is the *envelope*—the four laws the artifact satisfies that no stock exploration policy does.

3.2 The four laws

Invariant 1 (Deterministic replay). *For identical inputs, the allocator produces bit-identical outputs on every host, and the output carries a canonical hash: candidates are ordered by a*

canonical `candidate_hash` before any arithmetic, and all arithmetic is integer fixed-point. Inspection is not enough; the cross-host golden fixture of Chapter 4 is the executed proof.

Invariant 2 (Conservation by construction). *Discrete budgets are assigned by floor projection: $\sum_i \lfloor p_i B \rfloor \leq B \sum_i p_i = B$, with the residual one-sided and bounded by the number of active branches. The certified kernel’s own documentation goes further than we initially did: it measured its residuals across adversarial fixtures before claiming the bound, and explicitly narrowed an earlier “leaves sum to one” claim after finding a legitimate exception. Conservation is a property of the projection, verified on prime budgets (e.g., $104,726 \leq 104,729$ ticks over an 8-branch frontier, residual within the bound).*

Invariant 3 (Typed refusal). *Every failure mode refuses with a named type rather than degrading: cardinality beyond the compiled $N=8$ shape, non-finite features, a forced-but-absent transport, a mis-pointed artifact path, numeric overflow, underflow of an escort weight under a negative lens. The canonical failure this law exists to prevent is the silent fallback: during this work, a test caught our own bridge silently falling through a mis-set environment variable to a different artifact; the law was tightened and the test pinned (the refusal now names the offending path).*

Invariant 4 (Receipt). *Every allocation plan is content-bound: a SHA-256 digest over a canonical projection of the plan. Receipts stream into OCEL 2.0 event logs, where they become ordinary process data—queryable, conformance-checkable, and counterfactually mutable (Chapter 5).*

3.3 What the envelope buys, and what it costs

The envelope buys composition: any host that can execute WebAssembly can serve the identical law, and any auditor with the receipt can replay the decision. It costs performance flexibility: there is no temperature knob. The compiled lens policy is what it is; changing exploration behavior means regenerating policy upstream (a governance act with provenance), not tuning a parameter at the call site. The empirical consequence is measured in Chapter 4: the compiled policy is substantially flatter than a tuned softmax. We regard this as the transistor-versus-filament trade in miniature—a capability exchanged for a guarantee—and we report it rather than tune it away.

Chapter 4

The Single-Kernel Law

4.1 From one engine per host to one kernel for all hosts

Before this work, the ecosystem contained a certified Rust kernel (`bcinr-cmca`), a subprocess CLI wrapping it, and—as a local convenience—a float-softmax reference implementation inside the Python package. The reference implementation was the standard failure waiting to happen: a second implementation of the measure that would drift, diverge in its floats, and eventually be trusted by someone who did not know it was a stand-in. The single-kernel law is the deletion:

There is exactly one implementation of the allocation measure in the ecosystem. It is the vendored, certified kernel. Every host obtains its measure from that kernel. There is no fallback implementation, reference or otherwise.

Enforcement is structural, not conventional: the Python allocator class no longer accepts an `engine` or `tau` argument at all (a Chicago test pins that reintroducing them raises `TypeError`), and the surviving transports—in-process WebAssembly first, CLI subprocess as availability fallback—are built from the same vendored commit (`2ae60cf0`), so transport choice cannot change an answer.

4.2 The WebAssembly deployment

The kernel is compiled to `wasm32-wasip1` as a cdylib exporting a three-function linear-memory ABI:

```
cmca_alloc(len) -> ptr
cmca_free(ptr, len)
cmca_rank_json(ptr, len) -> ptr ; [u32 LE out_len][JSON bytes]
```

The request/response JSON contract is the CLI’s, verbatim: `{“candidates”: [{“name”, “measures”[m0..m3]}]}in;{“ranking”:[{“name”, “share”}]} out`; refusals as `{“error”}` envelopes with the engine’s own message. The four caller-defined measure axes are the

branch’s option entropy, inverse cost, historical yield, and salience (3.1)—the same feature vector for every host, encoded once, in the bridge.

Two hosts were verified. CPython loads the artifact through `wasmtime`; the BEAM loads the *same artifact file* through `wasmex`, with trap-restart semantics (the guest builds with `panic=abort`, so a trapped instance is killed and rebuilt, its state never reused). The artifact is committed to the repository (421,988 bytes), so consumers need neither a Rust toolchain nor a subprocess binary—only a WASM runtime.

4.3 Bit-identity across hosts

The load-bearing empirical claim of this thesis is this table. One request—three candidates with fixed measure axes—was evaluated by the kernel through four paths: natively (CLI subprocess), in-process on CPython (`wasmtime`), in-process on the BEAM (`wasmex`), and the raw JSON compared across transports:

Candidate	Share (exact)	As binary fraction
beta	0.2135009765625	13,983/65,536
alpha	0.153564453125	10,062/65,536
gamma	0.1290130615234375	8,452/65,536

Table 4.1: Cross-host golden fixture. The shares are exact Q16.16 binary fractions; the BEAM test pins these values, captured through the CPython/`wasmtime` path over the same artifact. Any drift in the artifact, the transport, or the host binding fails the fixture by name.

The payloads are byte-identical JSON across all four paths. This is not an approximation claim to be argued; it is an equality claim to be tested, and the test is in the repository. To our knowledge, the deployment of one certified fixed-point policy kernel serving Python and Erlang hosts bit-identically through WebAssembly, with the agreement pinned by golden fixtures, is a construction not previously documented in the agent-governance literature.

4.4 The honest finding: flatness

When the tunable softmax was deleted, the benchmarks that swept its inverse temperature were refactored to drive the shipped kernel. The measured behavior of the compiled lens policy is markedly flatter than a high- τ softmax. In a scenario constructed to contain one viable pair and one dead-end candidate (entropy 0.01, yield 0.01, cost 100), the compiled policy allocates:

The interpretation offered to the reader is deliberately double-edged. As a performance instrument, the compiled policy leaves exploration budget on the table by the standards of a tuned softmax. As a governance instrument, its flatness is honest: the policy refuses

Candidate	Measured share
b_promising	0.490498
b_promising_alt	0.340867
b_dead_end	0.168635

Table 4.2: The compiled lens policy funds every candidate with real mass; the deleted softmax at $\tau=5$ starved the dead end entirely. The pruning cutoff is therefore a projection-level parameter calibrated to measured shares, documented in the test that pins it.

to silently murder a candidate someone lawfully proposed, and forces the kill decision into the projection layer where it is receipted. Which of these framings matters is precisely the question the thesis exists to re-pose.

Chapter 5

Process Law and Counterfactual Conformance

5.1 Allocations as events

An allocation plan that ends its life in application memory is ungoverned. The receipted plan is projected into OCEL 2.0 logs as events bound to the objects they concern (plans, branches, worlds), alongside the actuation receipts of the actions the allocation authorized. From that moment the allocation is process data: queryable with object-centric query languages, joinable across the lifecycles of interacting agents, and discoverable as process models.

5.2 Conformance and counterfactual mutation

Conformance checking against discovered process models (alignments [10]) validates that executed lifecycles obey the process law. The counterfactual battery goes further: programmatic mutants—premature actuation, unadmitted action injection, skipped teardown, subtask inversion, crossed object links, resource-allocation inversion—must each be *detected* by the governed pipeline, and the detection is tested. The resource-allocation mutant is the one this thesis adds to the literature’s usual set: it inverts the allocator’s own output (assigning the strongest branch the weakest budget) and requires that the inversion fail conformance downstream. A governor whose decisions cannot be counterfactually falsified is decoration; the battery is the difference.

5.3 The limitation we declare

The mutators are generated by the same codebase that generates the defenses. Known-mutation testing is strong regression discipline but weak adversarial evidence; it certifies the fence against the adversaries the fence-builder imagined. Chapter 6 records this as an open standing item rather than a resolved one.

Chapter 6

The Falsification Ledger

6.1 Deletion as a scholarly act

A thesis that only accumulates claims trains its author to defend the indefensible. This manuscript’s methodological core is the opposite discipline: every claim is adversarially audited, and claims that fail are *deleted with receipt*—removed from the codebase in named commits, recorded here, and cited as evidence that the audit was real. The ledger also enforces a monotone law borrowed from the laboratory’s operating contract: the hand-written surface must shrink; growth of the hand-written surface requires, in the same change, a paydown plan.

6.2 The autopsies

6.2.1 The circular benchmark

An earlier “empirical value analysis” compared CMCA against a greedy baseline defined as argmax on immediate yield—an allocator blind, by construction, to the option-entropy term in the evaluation metric, which was in turn the allocator’s own objective. The scenario was hand-tuned so the entropy-aware algorithm had to win; the comparator’s arm existed only as comments. Audit verdict: a characterization test masquerading as a comparative experiment. Deleted as evidence; retained (rewritten) as a characterization of the shipped engine, with its thresholds calibrated to measured shares and its numbers documented in-test (Table 4.2).

6.2.2 The theorems that held for every softmax

Four “theorems” pinned properties of the softmax measure: floor conservation ($\sum [p_i B] \leq B$, one line from the definition of floor), entropy monotonicity ($dH/d\tau = -\tau \text{Var}(S)$, an exponential-family identity), order preservation (softmax monotonicity), and permutation invariance (sorting commutes). Audit verdict: true, elementary, and non-discriminating—they distinguished CMCA from no softmax allocator whatsoever. The test file was deleted

with the reference engine (commit `c1d04ffc`).

6.2.3 The notational equivalence

A formal “proof of equivalence” between the Python softmax and the kernel’s escort distribution reduced, on inspection, to the observation that $m = e^S$ turns $m^q / \sum m^q$ into $e^{qS} / \sum e^{qS}$: a change of notation, verified numerically to 10^{-6} , which confirmed only that floating-point arithmetic worked. Deleted. The equivalence that matters was rebuilt as the bit-identity fixture of Table 4.1, which compares real builds of real transports rather than formulas with themselves.

6.2.4 The reference implementation itself

The final and largest entry: the local float-softmax engine, its temperature parameter, its salience method, and its end-to-end plumbing (plan fields, port contracts, cross-language resource attributes) were deleted outright once the single-kernel law held. The deletion removed approximately a thousand lines of hand-written measure code and every test that existed only to pin its behavior. What remains in the Python package is a projection: ordering, pruning, renormalization, anti-starvation, discrete assignment—all engine-agnostic, all tested against the kernel.

6.3 What the ledger buys

The ledger converts the thesis’s central risk—that a governance claim is itself an unaudited assertion—into structure. The reader does not have to believe the artifact is honest; the reader can observe that dishonesty would have required *surviving* an audit process that demonstrably deleted four of its own claims, including its own reference implementation. The ledger is, in the strict sense, the thesis’s own governance envelope applied to itself.

Chapter 7

The Phase-Change Analysis

7.1 A model of phase change

We adopt the following structural model, synthesized from the economic history of general-purpose technologies [4, 11, 6]. A technology undergoes a phase change when three conditions hold jointly:

1. **Constraint-class removal.** An entire *class* of failures or limitations of the old regime is removed structurally—not reduced, not mitigated, but made impossible by construction.
2. **Composition unlock.** The removal enables assemblies of a kind and scale that the old regime could not reach at any price, because the old failure rate grew with the size of the assembly.
3. **A compounding interface.** A standardized interface causes improvement effort to accumulate onto the new regime rather than forking across variants.

7.1.1 Steam

The steam engine did not outrun good water mills on efficiency at introduction; early engines were thermally poor and economical only where fuel was free (minehead pumping). The phase change was the removal of the *biological and geographical* constraint class: power that did not tire, did not freeze, and did not depend on river siting. The composition unlock followed—factories could locate anywhere, and the factory system emerged as an assembly of powered stations. The compounding interface was standardization (interchangeable parts, standard threads), which turned power into a commodity input that improved system-wide when any engine improved.

7.1.2 The transistor

At demonstration in 1947–48 [2], the transistor was not faster than the vacuum tube. Its initial superiority was the removal of the *filament failure class*: no thermal wear-

out, no warm-up, lower power. The composition unlock is the heart of the story: a machine with n components of per-component reliability r has availability r^n , and ENIAC-scale tube assemblies were already failing continuously; million-component machines were arithmetically impossible in tubes. The removed failure class is what made large assemblies thinkable, and integration [7] is the compounding interface—the planar process—onto which fifty years of improvement effort accumulated.

7.2 The mapping

	Old regime: implementation policies	This work: single-kernel governance
Constraint class removed	Divergent implementations; float non-portability; silent fallbacks; invisible decisions	One certified kernel; integer fixed-point replay; typed refusal; receipted events
Composition unlock	Audit surface grows per implementation; cross-language delegation unverifiable	Any WASM host serves the identical law; receipted chains cross language boundaries
Compounding interface	Tuning effort forks across variants	Improvement lands in the kernel/ontology and propagates by artifact refresh with provenance tripwires
Legacy metric at transition	(Steam: efficiency) (Tubes: speed)	(Here: exploration sharpness—the compiled policy is flatter, Table 4.2)

Table 7.1: The structural mapping. Each row of the old-regime column names a failure class the new regime removes by construction; the final row records that the new regime was, at transition, worse on the old regime’s headline metric—as were steam and the transistor.

The composition argument deserves its quantitative form. Suppose each host-level implementation of a policy has probability p of divergence per deployment-relevant change (platform arithmetic, version drift, silent fallback paths). An audit over H hosts that must agree has integrity $(1 - p)^H$, which decays exponentially: per-implementation governance cannot scale to many hosts, exactly as tube machines could not scale to many components. The single-kernel law replaces $(1 - p)^H$ with a single binary question—do the pinned fixtures pass—whose cost is constant in H . This is the availability-curve argument of the transistor, transposed from components to hosts.

7.3 Disanalogies, stated

The analogy fails in three ways a reviewer should press on, and we state them rather than leave them to be found.

Scale and adoption. Steam and the transistor were adopted by economies over decades; this work is one laboratory ecosystem. The claim demonstrated is *mechanism*, not *history*. Whether the field reorganizes around manufactured governance is an empirical question this thesis does not answer.

The binding-constraint question. We assert that auditability, not performance, binds trustworthy autonomy. For many deployed agent systems today this is arguably false—their operators accept unauditable allocation in exchange for results. The thesis’s claim is that this exchange becomes unavailable as consequence scales, which is a prediction, not a measurement.

The kernel is small. The certified kernel allocates over eight candidates with a compiled lens policy. Steam powered mills; the transistor scaled to billions. The honest statement is that the *pattern*—certified kernel, pinned cross-host identity, deletion of local reimplementations, receipts into process law—is demonstrated end-to-end on a size where every element is real, and the pattern’s scaling is future work with named obstacles (lens-policy expressiveness at scale, wasm artifact governance across supply chains, the known-mutation limitation of the counterfactual battery).

7.4 Falsifiers

The phase-change framing would be falsified, locally, by any of: (i) a demonstration that per-implementation governance scales across heterogeneous hosts without a single-kernel discipline (the composition argument would then be unnecessary); (ii) divergence between the certified kernel’s outputs across hosts that survives the fixture tests (the central empirical claim would be false); (iii) deployment evidence that receipted, conserving, deterministic allocation *underperforms* per-implementation tuning so severely that operators refuse the guarantee at consequence scales where auditability matters (the trade would be priced wrong); or (iv) an advance that makes verification of arbitrary floating-point policy implementations free, removing the constraint class by other means.

Chapter 8

Conclusion

This thesis set out to answer a question disguised as a complaint: after the adversarial audit stripped away the circular benchmarks, the trivial theorems, the notational equivalences, and finally the reference implementation itself, what was left? The answer is an envelope and a law. The envelope—deterministic replay, conservation by construction, typed refusal, receipt into process data—is what consequential allocation must satisfy before autonomy can be delegated across trust boundaries. The law—one certified kernel, served bit-identically to every host through WebAssembly, with the local reimplementations deleted rather than deprecated—is what makes the envelope cheap to verify: constant in the number of hosts, pinned by fixtures that fail by name.

The phase-change claim was argued structurally and scoped honestly: the same three conditions that turned the steam engine and the transistor into regime boundaries—constraint-class removal, composition unlock, compounding interface—hold in the small for this artifact chain, with the characteristic signature that the new regime is initially worse on the old regime’s metric. Whether that structure compounds beyond one laboratory is the open question the artifact chain is positioned to answer, and the falsifiers in Chapter 7 are the terms of its answering.

The deepest contribution may be the ledger. A discipline that deletes its own weaker claims, with receipts, is one that can be trusted to report its stronger ones. Governance, in the end, is not a property an artifact has. It is a property of the process that is allowed to modify the artifact—and this thesis has tried to be an instance of its own thesis about it.

Appendix A

Artifact and Verification Map

All quantitative claims in the text anchor here. Repositories: `seanchatmangpt/autofde-lab` (branch `feat/semantic-model-manufacturing`) and the sibling `ash_autofde` project (local).

Commit	Role	Content
39d244a9	Vendoring	bcinr-cmca crate vendored; kernel becomes default engine
465616a3	WASM transport	shim crate, committed artifact, wasm-first dispatch, 7 Chicago tests
3e2067f9	Receipt	verification receipt with full command/exit ladder
c1d04ffc	Deletion	reference-softmax engine, tau, and its tests deleted; single-kernel law
11e0559	BEAM host	<code>ash_autofde</code> initial commit; wasmex engine + golden fixture
1617520	BEAM update	tau_temperature attribute removed to match plan contract

Table A.1: Commit anchors for the claims in this thesis.

Key artifacts: vendored kernel at `vendor/bcinr` (pinned `2ae60cf0`; provenance in `VENDORING.md`); WASM shim at `wasm/bcinr-cmca-wasm` with committed artifact `wasm/artifacts/bcinr_cmca_wasm.wasm` (421,988 bytes; exports `cmca_alloc`, `cmca_free`, `cmca_rank_json`); Python transport `src/autofde_lab/cmca/bcinr_wasm.py`; BEAM host `lib/ash_autofde/wasm_cmca.ex`; projection-only allocator `src/autofde_lab/cmca/cascade.py`; golden fixture test `test/ash_autofde_wasm_cmca_test.exs`; bit-identity and law-pinning tests under `tests/cmca/`.

Verification status at writing: 35 tests across `tests/cmca` and `tests/beam` pass; consumer suites (OCEL counterfactual engine, autodev loop, GymAct/OCEL ecosystem) pass; `ash_autofde mix test` passes 10 tests and 1 doctest; lint clean on all touched files. Standing limitation: the counterfactual battery’s mutators are generated by the codebase they test (Chapter 5).

Appendix B

The Deleted Claims, For the Record

For transparency, the headlines that did not survive audit, in their own terms: “mathematically rigorous and empirically sound” (the theorems were one-line properties of any softmax); “proved and verified equivalence” (a change of notation); “empirical value analysis vs. alternatives” (a comparator that existed only in comments); “optimal collapse risk, maximal option preservation” (assertion, not evidence). Their deletion commits are the citations. The reader is invited to distrust, on this basis, exactly nothing less than everything else in this manuscript—and to check.

Bibliography

- [1] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time Analysis of the Multiarmed Bandit Problem,” *Machine Learning*, 47(2–4):235–256, 2002.
- [2] J. Bardeen and W. H. Brattain, “The Transistor, A Semi-Conductor Triode,” *Physical Review*, 74(2):230–231, 1948.
- [3] D. Goldberg, “What Every Computer Scientist Should Know About Floating-Point Arithmetic,” *ACM Computing Surveys*, 23(1):5–48, 1991.
- [4] D. S. Landes, *The Unbound Prometheus: Technological Change and Industrial Development in Western Europe from 1750 to the Present*. Cambridge University Press, 1969.
- [5] D. MacKenzie, *Mechanizing Proof: Computing, Risk, and Trust*. MIT Press, 2001.
- [6] J. Mokyr, *The Lever of Riches: Technological Creativity and Economic Progress*. Oxford University Press, 1990.
- [7] G. E. Moore, “Cramming More Components onto Integrated Circuits,” *Electronics*, 38(8), 1965.
- [8] A. F. Ghahfarokhi, G. Park, A. Berti, and W. M. P. van der Aalst, “OCEL (Object-Centric Event Log) 2.0,” 2022. †
- [9] K. Küsters and W. M. P. van der Aalst, “Object-Centric Process Querying,” 2025. †
- [10] A. Berti, S. J. van Zelst, and W. M. P. van der Aalst, “Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science,” 2019. †
- [11] N. Rosenberg, *Perspectives on Technology*. Cambridge University Press, 1976.
- [12] C. Rosin, “Multi-Armed Bandits with Episode Context,” *Annals of Mathematics and Artificial Intelligence*, 61(3):203–230, 2011. †
- [13] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press, 2018.

Bibliographic note. Entries marked † are provisional to this manuscript draft and must be verified against library records (volumes, pages, exact author lists) before any submission or deposit. The artifact citations (commits, files, measurements) are exact and reproducible from the repository at the named commits.